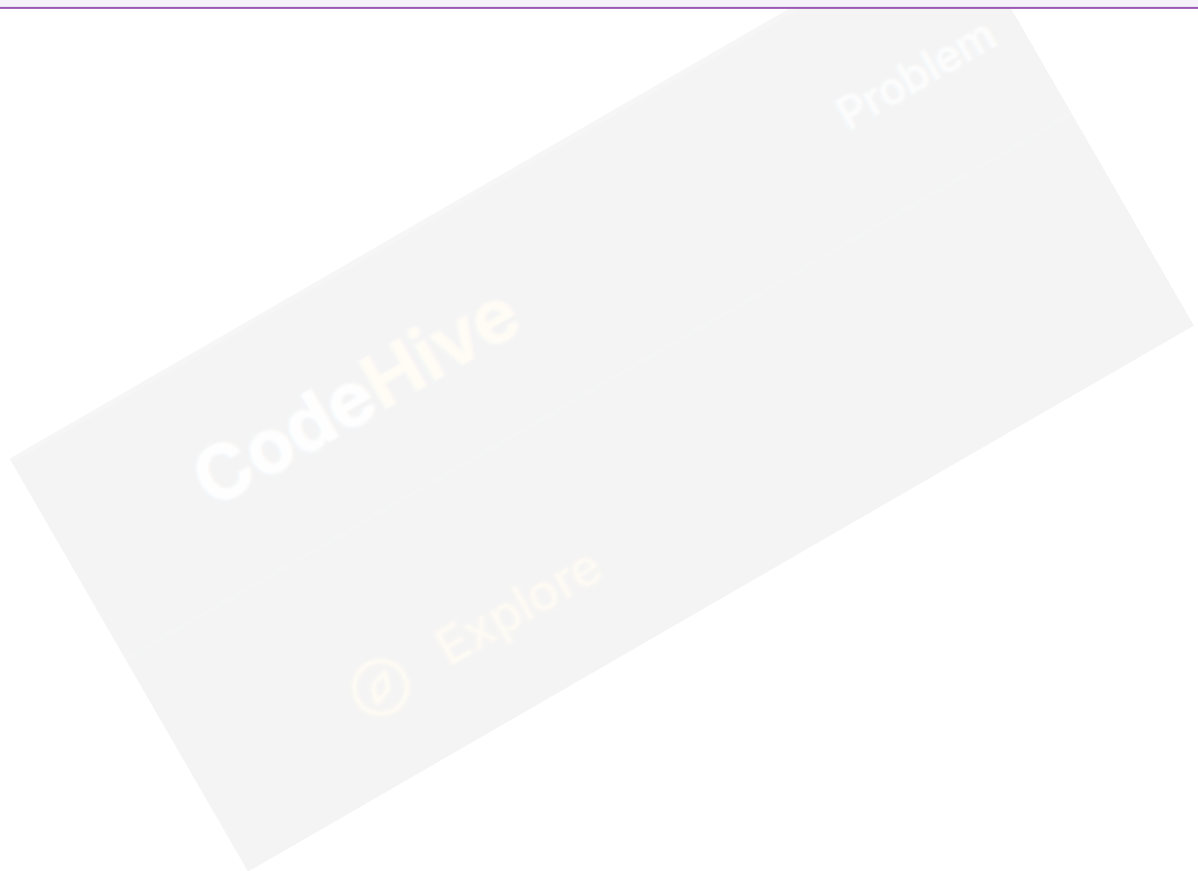


C Error Challenges

Error challenges in C are programming tasks where you must identify, debug, and fix errors in given code. They test your deep understanding of language rules, logic, and runtime behavior.

CodeHive Strategy: Don't just look for "red lines"—thinking like a compiler is the key to solving these challenges.



Challenge 1: The Syntax Hunt

Identify the syntax errors in this snippet:

```
#include <stdio.h>
int main() {
    int x = 10
    printf("Value is %d", x)
    return 0
}
```

Solution: Missing semicolons after variable declaration, printf, and return statement. Syntax errors are compile-time stoppers.

CodeHive

@ Explore

Challenge 2: The Crash Course

What happens when this program runs?

```
int main() {  
    int a = 10, b = 0;  
    int res = a / b;  
    printf("%d", res);  
    return 0;  
}
```

Solution: *Floating point exception (core dumped).* Division by zero is a classic runtime error that causes immediate termination.

CodeHive

Ⓢ Explore

Challenge 3: The Logic Trap

The output is "Equal" even when x is 5 and y is 10. Why?

```
int x = 5, y = 10;
if (x = y) {
    printf("Equal");
} else {
    printf("Not Equal");
}
```

Solution: Using = (assignment) inside the if condition instead of == (comparison).

CodeHive

@ Explore

Challenge 4: The Dangling Danger

Identify the dangerous practice below:

```
int *ptr = malloc(sizeof(int));
*ptr = 100;
free(ptr);
printf("%d", *ptr);
```

Solution: Accessing memory after it has been freed. This creates a **Dangling Pointer**, leading to unpredictable crashes.

Challenge 5: Out of Bounds

Spot the error in array handling:

```
int arr[5] = {1, 2, 3, 4, 5};  
for(int i = 0; i ≤ 5; i++) {  
    printf("%d ", arr[i]);  
}
```

Solution: The loop condition `i ≤ 5` attempts to access `arr[5]`, which is out of bounds (arrays go 0 to 4).

CodeHive

Ⓢ Explore

Challenge 6: Type Mismatch

```
int main() {  
    int a = "10";  
    printf("%d", a);  
}
```

Solution: Assigning a string literal (address) to an integer variable. This is a semantic error—the grammar is fine, but the meaning is invalid.

CodeHive

@ Explore

Problem

7. Skills Tested ★

- **Trace Ability:** Following code line-by-line manually.
- **Memory Awareness:** Tracking life cycles of pointers.
- **Boolean Logic:** Understanding how C treats zero as false and non-zero as true.
- **Compiler Familiarity:** Knowing how to read GCC/Clang error logs.



8. Exam Strategy

When faced with a "Find the Error" question:

1. Check for semicolons and curly brace pairs.
2. Verify data types in `printf` format specifiers.
3. Look for uninitialized pointers.
4. Trace loops carefully for "Off-by-one" errors.



9. The Eternal Loop

```
for(int i = 10; i > 0; i++) {  
    printf("%d ", i);  
}
```

Question: Will this stop? **Answer:** Technically yes, when `i` overflows, but logically it's an infinite loop error.

CodeHive

@ Explore

Problem

10. Why it Matters

In the professional world, 70% of a developer's time is spent debugging. Mastering these challenges makes you a **Senior-level** thinker. Companies like Google and Microsoft use these exact types of challenges in interviews.



Key Points (Final Checklist)

- **Syntax:** Grammar mistakes (stops compilation).
 - **Runtime:** Execution mistakes (crashes program).
 - **Logical:** Thinking mistakes (wrong output).
 - **Memory:** Pointer mistakes (leak/dangling).
-

Crush the Code - CodeHive Academy

CodeHive

@ Explore